

Informe de Validaciones - Perfum Lab System

Proyecto: Perfum Lab System

Fecha: 3 de agosto de 2026

Tipo de documento: README tecnico

Tema: Validaciones de datos correctos, completos y con formato adecuado

1. Resumen

Se aplicaron validaciones mas completas en el sistema Perfum Lab para asegurar que los datos ingresados sean correctos, completos y mantengan el formato esperado antes de ser guardados o usados en procesos importantes.

Las validaciones fueron aplicadas en:

- Productos.
- Categorías.
- Inventario.
- Ventas.
- Facturas.
- Reportes.
- Exportaciones TXT, PDF y CSV.

El objetivo principal fue evitar registros incompletos, valores invalidos, cantidades incorrectas, fechas mal formateadas y archivos exportados con extensiones equivocadas.

2. Objetivo

El objetivo de esta mejora fue reforzar la calidad de los datos del sistema.

Ahora el sistema valida que:

- Los campos obligatorios no esten vacios.
 - Los textos tengan una longitud adecuada.
 - Los nombres importantes contengan letras.
 - Los SKU tengan un formato correcto.
 - Las cantidades sean numeros enteros validos.
 - Los precios y costos sean numeros validos y no negativos.
 - Los IDs sean positivos.
 - Las fechas tengan formato **YYYY-MM-DD**.
 - Las rutas de exportacion tengan la extension correcta.
-

3. Archivo central de validaciones

Se creo el archivo:

```
app/validaciones.py
```

Este archivo centraliza reglas reutilizables para todo el sistema.

Funciones principales:

Funcion	Proposito
<code>validar_sku</code>	Valida que el SKU tenga longitud correcta y solo use letras, numeros, guiones o guion bajo.
<code>validar_nombre_cliente</code>	Valida que el cliente tenga nombre obligatorio, longitud adecuada y al menos una letra.
<code>validar_decimal_no_negativo</code>	Valida costos y precios numericos, finitos y no negativos.
<code>validar_entero_no_negativo</code>	Valida valores enteros desde cero, como stock.
<code>validar_entero_positivo</code>	Valida cantidades mayores que cero.
<code>validar_id_positivo</code>	Valida IDs de producto, venta, factura o usuario.
<code>validar_fecha_iso</code>	Valida fechas en formato <code>YYYY-MM-DD</code> .
<code>validar_rango_fechas</code>	Valida que la fecha inicial no sea mayor que la fecha final.
<code>validar_ruta_exportacion</code>	Valida que los archivos exportados tengan la extension esperada.

4. Validaciones en productos

Archivo trabajado:

```
app/models/producto.py
```

Se reforzo el metodo:

```
Producto.validar()
```

Validaciones aplicadas:

- SKU obligatorio.
- SKU minimo de 3 caracteres.
- SKU maximo de 30 caracteres.
- SKU solo con letras, numeros, guiones y guion bajo.
- Nombre obligatorio.
- Nombre minimo de 2 caracteres.
- Nombre maximo de 120 caracteres.

- Nombre debe contener al menos una letra.
- Marca opcional, maximo 80 caracteres.
- Descripcion opcional, maximo 500 caracteres.
- Costo numerico, finito y no negativo.
- Precio numerico, finito y no negativo.
- Stock actual entero y no negativo.
- Stock minimo entero y no negativo.

Ejemplos rechazados:

```
SKU: AB
SKU: SKU CON ESPACIO
Nombre: 1
Precio: NaN
Stock actual: -5
```

5. Validaciones en categorias

Archivo trabajado:

```
app/models/categoria.py
```

Validaciones aplicadas:

- Nombre obligatorio.
- Nombre minimo de 2 caracteres.
- Nombre maximo de 80 caracteres.
- Nombre debe contener al menos una letra.
- Descripcion opcional, maximo 250 caracteres.

Ejemplos rechazados:

```
Nombre vacio
Nombre: 1
Nombre demasiado largo
```

6. Validaciones en inventario

Archivo trabajado:

```
app/controllers/inventario_controller.py
```

Funciones reforzadas:

```
registrar_entrada()  
registrar_salida()  
registrar_ajuste()
```

Validaciones aplicadas:

- ID de producto positivo.
- Cantidad de entrada mayor que cero.
- Cantidad de salida mayor que cero.
- Nuevo stock de ajuste entero y no negativo.
- Motivo opcional con longitud maxima de 160 caracteres.
- La salida no puede superar el stock disponible.
- El ajuste debe cambiar realmente el stock actual.

Ejemplos rechazados:

```
producto_id = 0  
cantidad = 0  
cantidad = -3  
cantidad = 1.5  
nuevo_stock = -1  
salida mayor al stock disponible
```

7. Validaciones en ventas

Archivo trabajado:

```
app/ventas/ventas.py
```

Funciones reforzadas:

```
registrar_venta()  
registrar_venta_multiple()  
anular_venta()
```

Validaciones aplicadas:

- Nombre del cliente obligatorio.
- Nombre del cliente minimo de 2 caracteres.
- Nombre del cliente maximo de 120 caracteres.
- Nombre del cliente debe contener al menos una letra.

- ID de usuario positivo.
- ID de producto positivo.
- Cantidad entera y mayor que cero.
- Productos del carrito con estructura valida.
- Stock suficiente antes de registrar venta.
- ID de venta positivo para anulaciones.
- No se puede anular una venta que ya tiene factura.

Ejemplos rechazados:

```
Cliente vacio
Cliente: 12345
producto_id = 0
cantidad = 0
cantidad = 1.5
cantidad mayor al stock disponible
venta ya facturada
```

8. Validaciones en facturas

Archivo trabajado:

```
app/facturas/facturas.py
```

Funciones reforzadas:

```
generar_factura()
obtener_factura_por_id()
obtener_factura_por_venta_id()
obtener_detalle_factura()
generar_texto_factura()
exportar_factura_txt()
exportar_factura_pdf()
```

Validaciones aplicadas:

- ID de venta positivo para generar factura.
- ID de factura positivo para consultar detalle.
- No se puede facturar una venta inexistente.
- Solo se pueden facturar ventas completadas.
- No se puede generar factura duplicada para la misma venta.
- Exportacion TXT debe usar extension **.txt**.
- Exportacion PDF debe usar extension **.pdf**.
- La carpeta destino debe existir.

Ejemplos rechazados:

```
venta_id = ABC
venta inexistente
venta anulada
factura duplicada
exportar PDF como factura.txt
exportar TXT como factura.pdf
```

9. Validaciones en reportes

Archivo trabajado:

```
app/reportes/reportes.py
```

Funciones reforzadas:

```
obtener_resumen_reportes()
obtener_productos_mas_vendidos()
obtener_ventas_recientes()
exportar_csv()
```

Validaciones aplicadas:

- Fecha de inicio con formato **YYYY-MM-DD**.
- Fecha de fin con formato **YYYY-MM-DD**.
- La fecha de inicio no puede ser mayor que la fecha de fin.
- Las fechas de registros deben ser validas.
- Exportacion CSV debe usar extension **.csv**.
- El CSV debe tener columnas definidas.
- Cada columna debe tener encabezado.
- Cada fila exportada debe contener las columnas requeridas.

Ejemplos rechazados:

```
2026/08/03
03-08-2026
fecha_inicio = 2026-12-31 y fecha_fin = 2026-01-01
exportar CSV como reporte.txt
columnas sin encabezado
filas incompletas
```

10. Pruebas QA agregadas

Se amplio la suite de QA en:

```
tests/test_qa_core.py
```

Nueva prueba agregada:

```
test_validaciones_de_formato_y_campos_requeridos
```

Esta prueba valida que el sistema rechace:

- Productos con SKU incorrecto.
- Productos con nombre incompleto.
- Productos con precio invalido.
- Categorías sin nombre valido.
- Ventas con IDs invalidos.
- Ventas con cliente incorrecto.
- Ventas con cantidades decimales.
- Facturas con ID invalido.
- Reportes con fechas incorrectas.
- Rangos de fechas incorrectos.
- Exportaciones con extension equivocada.

11. Resultado de pruebas

Comando usado:

```
uv run python -m unittest discover -s tests
```

Resultado obtenido:

```
.....
-----
Ran 7 tests in 1.295s

OK
```

Tambien se ejecuto el smoke test:

```
uv run python probar_ventas_facturas.py
```

Resultado:

```
QA smoke OK  
Venta generada: #1 | Total: L 2480.00  
Factura generada: FAC-000001
```

12. Matriz de validaciones

Area	Validacion	Estado
Productos	SKU obligatorio y con formato correcto	Aplicado
Productos	Nombre completo y con letras	Aplicado
Productos	Costo y precio no negativos	Aplicado
Productos	Stock entero y no negativo	Aplicado
Categorias	Nombre obligatorio y valido	Aplicado
Inventario	IDs y cantidades validas	Aplicado
Inventario	Salida no mayor al stock disponible	Aplicado
Ventas	Cliente obligatorio y valido	Aplicado
Ventas	Cantidades enteras mayores que cero	Aplicado
Ventas	Stock suficiente	Aplicado
Ventas	Bloqueo de anulacion si existe factura	Aplicado
Facturas	Venta existente y completada	Aplicado
Facturas	Bloqueo de factura duplicada	Aplicado
Exportaciones	Extension correcta por tipo de archivo	Aplicado
Reportes	Fechas con formato YYYY-MM-DD	Aplicado
Reportes	Rango de fechas valido	Aplicado
CSV	Columnas, encabezados y filas completas	Aplicado

13. Como convertir este Markdown a PDF

Este archivo esta listo para exportarse a PDF.

Archivo:


```
docs/README_VALIDACIONES.md
```

Con Pandoc:

```
pandoc docs\README_VALIDACIONES.md -o docs\README_VALIDACIONES.pdf
```

Con Visual Studio Code:

1. Abrir `docs/README_VALIDACIONES.md`.
 2. Usar una extension como `Markdown PDF`.
 3. Ejecutar `Markdown PDF: Export (pdf)`.
-

14. Conclusion

Las validaciones aplicadas ayudan a proteger la integridad del sistema Perfum Lab.

Ahora los datos principales pasan por controles mas estrictos antes de guardarse o procesarse. Esto reduce errores de captura, inconsistencias en inventario, ventas invalidas, facturas incorrectas y reportes con fechas o archivos mal formateados.